MAANG JavaScript Prep: Deep Dive

Section 1: Core Theory (1–4)

1. Event loop and asynchronous execution

JavaScript is single-threaded. It uses a **Call Stack** to execute synchronous code line by line. When an asynchronous operation (like a timer or network request) occurs, it is handed off to the browser's Web APIs.

Once finished, callbacks are placed in queues. **Promise callbacks** (from `.then()`) go to the **Microtask Queue**, while **setTimeout callbacks** go to the **Task Queue** (Macrotask Queue). The Event Loop constantly checks: if the Call Stack is empty, it processes *all* microtasks first. Only when the Microtask Queue is completely empty will it process a single task from the Task Queue. This is why Promises always execute before timers, even if both resolve instantly.

2. Promise executor and states

A Promise has three states: **Pending** (ongoing), **Fulfilled** (success), and **Rejected** (failure).

The executor function (the function passed into `new Promise((resolve, reject) => {...})`) executes **synchronously** right away. It does not wait in a queue. If you throw an error synchronously inside the executor, the Promise automatically catches it and changes its state to Rejected.

3. Promise resolution procedure

When you call `resolve(value)`, the outcome depends on the `value`:
• **Primitive (e.g., number, string):** The Promise fulfills immediately with that value.
• **Another Promise:** The outer Promise "locks in" to the inner Promise, adopting its eventual fulfillment or rejection state.
• **Thenable (object with a .then method):** It treats it like a Promise, queuing a microtask to evaluate it safely.
• **Itself:** It throws a `TypeError` because a Promise cannot wait for itself (infinite loop).

4. Promise chaining and flattening

Calling `.then()` always returns a brand-new Promise. This is the foundation of chaining. If your `.then()` callback returns another Promise, the new Promise returned by `.then()` will "flatten" it—meaning it will pause and wait for your returned Promise to settle before passing the final value down the chain. If you forget to return a value in a `.then()`, JavaScript implicitly returns `undefined`, which is what the next step in the chain will receive.

Section 1: Core Theory (5–9)

5. Error propagation

Errors in a Promise chain skip over any subsequent `.then()` fulfillment handlers until they find a `.catch()` (or a `.then` with a rejection handler).

The `.finally()` block runs regardless of success or failure. Normally, `.finally()` is transparent—it passes the original value or error down the chain. However, if the code *inside* `.finally()` throws an error or returns a rejected Promise, that new error overrides the original outcome, and the chain becomes rejected with the new error.

6. Promise combinators differences

• **Promise.all():** Waits for all to fulfill. Rejects immediately if *any* single promise rejects. (Empty array fulfills immediately).
• **Promise.allSettled():** Waits for all to finish regardless of success/failure. Never rejects.
• **Promise.race():** Settles as soon as the first promise settles (success or fail). (Empty array hangs pending forever).
• **Promise.any():** Fulfills as soon as the first promise fulfills. Only rejects if *all* promises reject (AggregateError).

7. async and await internals

An `async` function always, implicitly, returns a Promise. When the engine encounters an `await` keyword, it immediately pauses the function's execution and returns control to the caller (outside the async function). Even if you `await` a Promise that is already fulfilled, JavaScript forces a suspension and queues a microtask to resume the function. It never continues synchronously.

8. Sequential vs. parallel execution

Use sequential `await` (one after another) only when task B requires the result of task A. This is slower because tasks wait in line. Use `Promise.all()` to fire multiple independent tasks at the same time (parallelly). This is much faster but uses more memory and network sockets simultaneously.

9. Async callbacks and loops

The standard array `forEach()` method does not understand Promises. It fires all callbacks synchronously without pausing for `await`. If you need tasks to run sequentially, use a standard `for...of` loop. If you want them to run parallelly, use `map()` combined with `Promise.all()`.

Section 1: Core Theory (10–12)

10. Promise cancellation & 11. Unhandled rejections

Promises represent operations that are already running; they don't have a built-in `.cancel()` method. To cancel network requests, modern JS uses `AbortController`, which sends a signal to abort the fetch.

An **unhandled rejection** occurs when a Promise fails but has no `.catch()` attached to handle the error. In Node.js, this is dangerous and causes the application process to crash in modern versions. Always attach catch blocks or use central error listeners.

12. Concurrency and memory

Concurrency is dealing with multiple things at once (e.g., the JavaScript event loop juggling multiple network requests), while **parallelism** is doing multiple things at the exact same physical time (e.g., using multi-core Web Workers).

Memory impact: Every pending Promise retains references to its callbacks and their associated closures (variables from the surrounding scope). If you launch 10,000 unbounded concurrent operations (e.g., mapping 10k fetch requests into `Promise.all()`), you immediately allocate memory for 10k Promise objects, their closures, and exhaust network sockets. This creates massive memory spikes and aggressive garbage collection pauses. This is why high-volume tasks must be bounded or processed in batches.

Section 2: Output-based questions (13–15)

13. Promise executor vs. .then()

```
console.log("A");
const promise = new Promise(resolve => {
  console.log("B");
  resolve("C");
});
promise.then(console.log);
console.log("D");
```

> **Output:** A, B, D, C
> **Why?** "A" logs synchronously. The Promise executor runs immediately, logging "B". `resolve("C")` registers the `.then()` callback into the microtask queue. The synchronous code finishes by logging "D". Finally, the call stack empties, the event loop picks up the microtask, and "C" is logged.

14. Promise reactions and timers

```
console.log(1);
setTimeout(() => console.log(2), 0);
```

Output Snippets (15–18)

15. Nested microtasks and timers

```
console.log("Start");
Promise.resolve().then(() => {
  console.log("A");
  Promise.resolve().then(() => console.log("B"));
  setTimeout(() => console.log("C"), 0);
});
Promise.resolve().then(() => console.log("D"));
console.log("End");
```

Output: Start, End, A, D, B, C
Why? "Start" and "End" run synchronously. First outer Promise queues microtask A. Second outer Promise queues microtask D. The event loop runs A, which queues nested microtask B and task C. Event loop continues to next outer microtask D. Then it processes the newly queued nested microtask B. Finally, tasks: C.

16. Promise chaining values & 17. Missing return

```
// Q16
Promise.resolve(5)
  .then(value => value * 2)
  .then(value => { console.log(value); return value + 1; })
  .then(value => console.log(value));

// Q17
Promise.resolve(10)
  .then(value => { Promise.resolve(value + 5); })
  .then(value => console.log(value));
```

Q16 Output: 10, then 11.
Q17 Output: undefined.
Why? In Q17, the first `.then()` block creates a new resolved Promise (15) but *fails to return it*. Because there is no `return` statement, the function implicitly returns `undefined`. The next `.then()` receives `undefined`.

18. Promise flattening

```
Promise.resolve(2)
  .then(value => Promise.resolve(value + 3))
  .then(value => Promise.resolve(Promise.resolve(value * 2)))
  .then(console.log);
```

Output Snippets (19–21)

19. Error propagation and recovery

```
Promise.resolve("A")
  .then(value => { throw new Error("Failure"); })
  .then(() => console.log("B"))
  .catch(error => { console.log(error.message); return "C"; })
  .then(console.log);
```

Output: "Failure", then "C"
Why? The thrown error skips the next `.then()` (so "B" is never logged) and falls directly into the `.catch()`. The catch block logs the error message ("Failure"). Because the catch block returns the string "C" normally without throwing, the Promise chain *recovers*, and the final `.then()` logs "C".

20. .finally() passing values

```
Promise.resolve(10)
  .finally(() => { console.log("Cleanup"); return 20; })
  .then(console.log);
```

Output: "Cleanup", then 10
Why? `.finally()` does not modify the passing value unless it throws an error. Returning 20 is completely ignored, and the original value 10 passes through.

21. .finally() throwing an error

```
Promise.reject(new Error("Original"))
  .finally(() => { throw new Error("Cleanup"); })
  .catch(error => console.log(error.message));
```

Output: "Cleanup"
Why? While `.finally()` usually passes outcomes through, if it *throws* an error, that new error overrides the original outcome. The catch block receives the "Cleanup" error, not the "Original" one.

Output Snippets (22–25)

22. Promise.all() order & 23. Promise.race()

```
// Q22
const a = new Promise(r => setTimeout(() => r("A"), 100));
const b = new Promise(r => setTimeout(() => r("B"), 10));
Promise.all([a, b]).then(console.log);

// Q23
const c = new Promise(r => setTimeout(() => r("C"), 50));
const d = new Promise((_, r) => setTimeout(() => r("D"), 10));
Promise.race([c, d]).then(console.log).catch(console.log);
```

Q22 Output: ["A", "B"]
Why? Even though "B" finishes in 10ms, Promise.all guarantees the output array matches the *input order* of the promises.

Q23 Output: "D" (via catch block)
Why? Race settles with the *first* promise to finish. "D" rejects in 10ms, which is faster than 50ms, so the entire race rejects.

24. Promise.any() mixed & 25. Empty Combinators

```
// Q24
Promise.any([Promise.reject("A"), Promise.resolve("B")])
  .then(console.log);

// Q25
Promise.all([]).then(v => console.log("all", v));
Promise.race([]).then(v => console.log("race", v));
```

Q24 Output: "B"
Why? Promise.any ignores rejections and returns the first fulfilled promise.

Q25 Output: "all []" (race never logs)
Why? `Promise.all([])` resolves instantly with an empty array. `Promise.race([])` stays pending *forever* because it has nothing to race.

Output Snippets (26–28)

26. Async function execution order

```
async function run() {
  console.log("A");
  await Promise.resolve();
  console.log("B");
}
console.log("C");
run();
console.log("D");
```

Output: C, A, D, B
Why? "C" runs synchronously. Calling `run()` starts it synchronously, logging "A". The `await` pauses `run()` and queues the rest of it as a microtask. Control returns globally, logging "D". Finally, the microtask runs, logging "B".

27. Sequential awaits

```
async function run() {
  await Promise.resolve();
  console.log(1);
  await Promise.resolve();
  console.log(2);
}
run();
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Output: 4, 1, 3, 2
Why? `run()` suspends at the first await. The outer Promise queues 3. 4 logs synchronously. Microtask queue has: [run pt2 (logs 1), outer promise (logs 3)]. 1 logs, which queues `run pt3` (logs 2). 3 logs. Then 2 logs.

28. Async rejection and try...catch

```
async function run() {
  try {
    Promise.reject(new Error("Failure"));
    console.log("Inside");
  } catch (error) {
    console.log("Caught");
  }
}
run();
```

Section 3: Design Questions (29–30)

29. Bounded-concurrency task processor

Architecture: You cannot use `Promise.all()` directly for 10,000 tasks as it executes them all concurrently, which will exhaust memory and connection pools. Instead, implement a "worker pool" or use an async semaphore pattern.

Create an array of size `N` (concurrency limit) holding "worker" functions. Each worker pulls the next available task from a shared queue, awaits it, saves the result at the original index (to preserve input order), and recursively calls itself to pull the next task until the queue is empty.

Error Handling & Retries: Wrap the fetch in a helper function that catches errors. If it fails, check the retry count. If retries remain, `await delay(Math.pow(2, attempt) * 100)` (exponential backoff) and try again. If it fails completely, return an error object instead of rejecting, ensuring the entire pool doesn't crash.

30. Resilient API aggregation service

Architecture: For a resilient aggregation layer (like a GraphQL resolver or BFF), you must use `Promise.allSettled()`, not `Promise.all()`. If one minor service fails in `Promise.all()`, the entire request rejects, destroying the valid data from other services.

Timeout/Cancellation: Wrap each service call in a `Promise.race()` against a timeout rejection. Better yet, pass an `AbortController.signal` into the `fetch()` call. If the timeout triggers, call `abort()` to physically close the network socket, saving server resources.

Deduplication: Use a memoization map during the lifecycle of the request. If Service A and Service B both request User profile #123, check if the Promise for that request already exists in the map. If so, return the existing Promise instead of firing a second network request.